

Recommendation System (Batch + Real-time)

Name: Ragini Singh

Roll No: 150096724023

Subject: System Design

1. Problem Statement

This project focuses on designing a scalable recommendation system similar to platforms like Netflix or Amazon. The system provides personalized recommendations based on user behavior such as clicks, views, and interactions.

The system combines:

- Batch Processing (historical data)
- Real-time Processing (recent user activity)

The goal is to generate accurate, fast, and relevant recommendations.

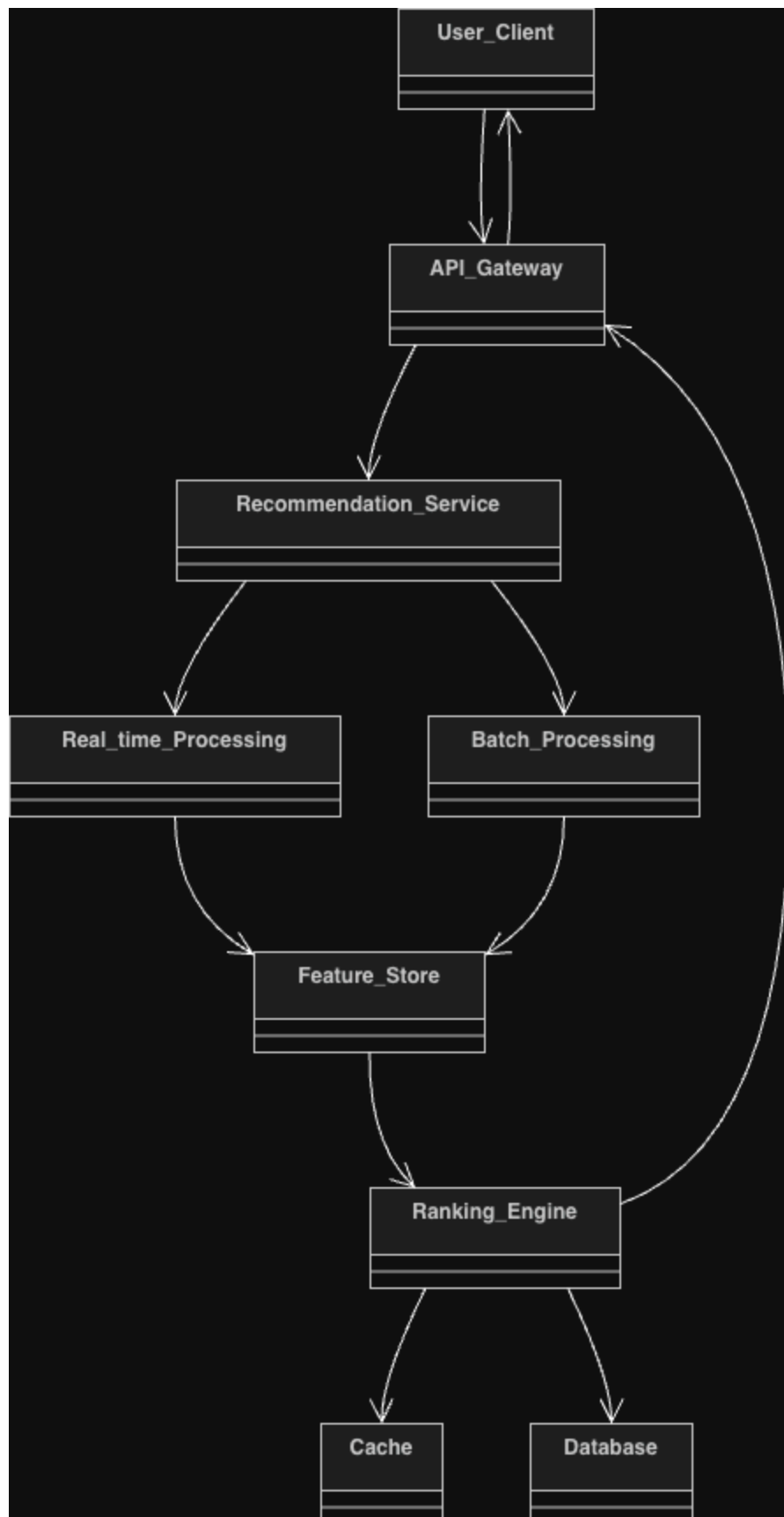
2. Solution Overview

The system uses a hybrid approach:

- Batch Layer → analyzes past user behavior
- Real-time Layer → captures recent actions
- Queue → handles event streaming
- Cache → improves speed
- Ranking Engine → generates final recommendations

This combination ensures both accuracy (batch) and freshness (real-time).

3. System Architecture



Explanation:

- Client sends request to backend
 - Backend processes request using recommendation service
 - Events are pushed into queue
 - Batch and real-time processing update recommendations
 - Data stored in database
 - Cache stores results for fast access
 - Response returned to user
-

4. Data Flow (Step-by-Step)

1. User performs an action (click/view/purchase)
 2. Request goes to backend server
 3. Event is pushed to queue (Kafka simulation)
 4. Real-time processor updates user preferences
 5. Batch processor analyzes historical data
 6. Ranking engine generates recommendation scores
 7. Cache stores results for fast retrieval
 8. Final response is sent back to user
-

5. Features Implemented

- Batch recommendation system
 - Real-time updates simulation
 - Queue-based event processing
 - Cache implementation for fast response
 - Ranking logic using popularity
-

6. Tech Stack

- Python (Google Colab)
 - Data Structures (Dictionary, List)
 - Queue Simulation (Kafka concept)
 - Cache (in-memory storage)
-

7. Code Explanation



```

BATCH PROCESSING

[22] def batch_recommend(user):
    watched = user_history.get(user, [])
    recommendations = []

    for item in item_popularity:
        if item not in watched:
            recommendations.append((item, item_popularity[item]))

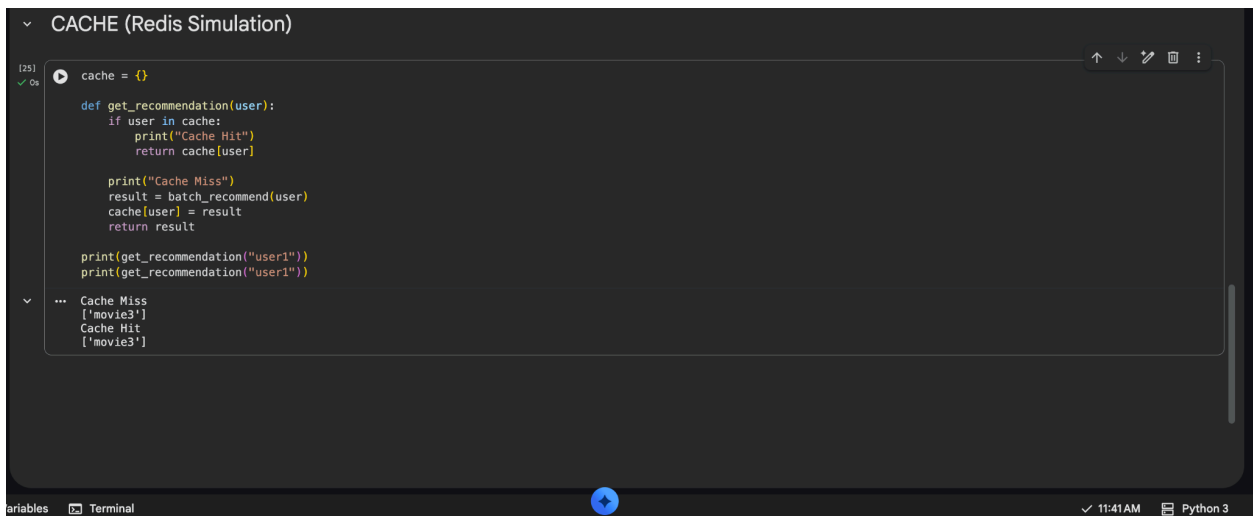
    recommendations.sort(key=lambda x: x[1], reverse=True)
    return [item for item, _ in recommendations[:3]]

print("Batch Recommendation:", batch_recommend("user1"))

... Batch Recommendation: ['movie4', 'movie3']

```

Figure 1: Batch Processing shows the batch processing layer where historical user interaction data is analyzed to generate baseline recommendations.



```

CACHE (Redis Simulation)

[25] cache = {}

def get_recommendation(user):
    if user in cache:
        print("Cache Hit")
        return cache[user]

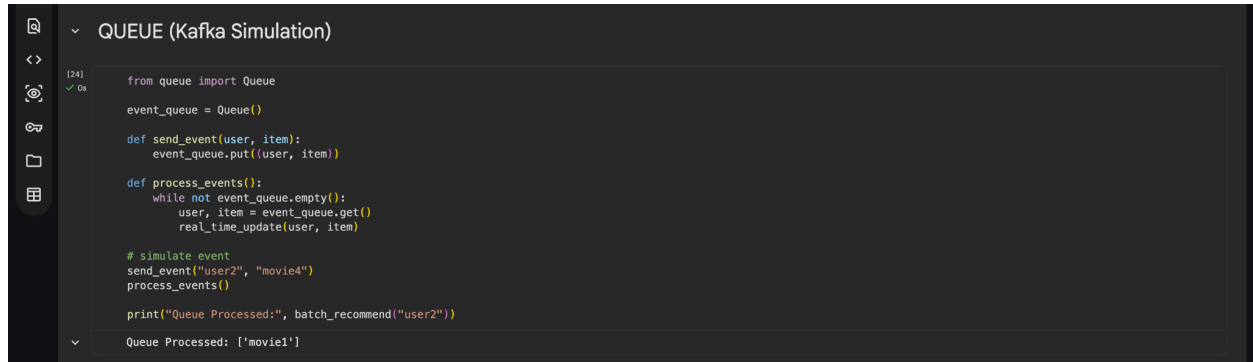
    print("Cache Miss")
    result = batch_recommend(user)
    cache[user] = result
    return result

print(get_recommendation("user1"))
print(get_recommendation("user1"))

... Cache Miss
['movie3']
Cache Hit
['movie3']

```

Figure 2 demonstrates the cache layer used to store frequently accessed recommendations for faster response time.



```

[24]
✓ Os
from queue import Queue

event_queue = Queue()

def send_event(user, item):
    event_queue.put((user, item))

def process_events():
    while not event_queue.empty():
        user, item = event_queue.get()
        real_time_update(user, item)

# simulate event
send_event("user2", "movie4")
process_events()

print("Queue Processed:", batch_recommend("user2"))

Queue Processed: ['movie1']

```

Figure 3 shows queue-based event processing where user actions are pushed into a simulated Kafka queue for asynchronous handling.



```

[23]
✓ Os
def real_time_update(user, new_item):
    if user not in user_history:
        user_history[user] = []

    user_history[user].append(new_item)

    if new_item in item_popularity:
        item_popularity[new_item] += 1
    else:
        item_popularity[new_item] = 1

# simulate real-time
real_time_update("user1", "movie4")

print("After Real-time Update:", batch_recommend("user1"))

After Real-time Update: ['movie3']

```

Figure 4 demonstrates the real-time processing layer that updates recommendations instantly based on recent user activity.

The image shows a Jupyter Notebook titled "Recommendation_System_Ragini.ipynb". The notebook is divided into two sections: "SAMPLE DATA" and "BATCH PROCESSING".

SAMPLE DATA

```
[21] In: # User history (batch data)
user_history = {
    "user1": ["movie1", "movie2"],
    "user2": ["movie2", "movie3"]
}

# Item popularity
item_popularity = {
    "movie1": 5,
    "movie2": 10,
    "movie3": 7,
    "movie4": 15
}
```

BATCH PROCESSING

```
[22] In: def batch_recommend(user):
        watched = user_history.get(user, [])
        recommendations = []

        for item in item_popularity:
            if item not in watched:
                recommendations.append((item, item_popularity[item]))

        recommendations.sort(key=lambda x: x[1], reverse=True)
        return [item for item, _ in recommendations[:3]]

        print("Batch Recommendation:", batch_recommend("user1"))

        Batch Recommendation: ['movie4', 'movie3']
```

The bottom of the notebook shows the status bar with "Variables", "Terminal", "11:41 AM", and "Python 3".

Figure 5 represents the overall system architecture including client, backend, queue, recommendation engine, database, and cache layers.

The image shows a Jupyter Notebook titled "Recommendation_System_Ragini.ipynb". The notebook is divided into two sections: "SAMPLE DATA" and "BATCH PROCESSING".

SAMPLE DATA

```
[21] In: # User history (batch data)
user_history = {
    "user1": ["movie1", "movie2"],
    "user2": ["movie2", "movie3"]
}

# Item popularity
item_popularity = {
    "movie1": 5,
    "movie2": 10,
    "movie3": 7,
    "movie4": 15
}
```

BATCH PROCESSING

```
[22] In: def batch_recommend(user):
        watched = user_history.get(user, [])
        recommendations = []

        for item in item_popularity:
            if item not in watched:
                recommendations.append((item, item_popularity[item]))

        recommendations.sort(key=lambda x: x[1], reverse=True)
        return [item for item, _ in recommendations[:3]]

        print("Batch Recommendation:", batch_recommend("user1"))

        Batch Recommendation: ['movie4', 'movie3']
```

The bottom of the notebook shows the status bar with "Variables", "Terminal", "11:41 AM", and "Python 3".

Figure 6 shows sample user history and item popularity data used for recommendation generation and ranking logic.

Recommendation_System_Ragini.ipynb

File Edit View Insert Runtime Tools Help

Commands Code Text Run all

Connect

Recommendation System (Batch + Real-time)

Name: Ragini Singh
Cohort : Jensen Huang
Roll No : 150096724023
Subject : System Design

Problem Statement

This project simulates a recommendation system using batch and real-time processing. It provides personalized recommendations based on user history and real-time updates.

SAMPLE DATA

```
1) # User history (batch data)
user_history = {
    "user1": ["movie1", "movie2"],
    "user2": ["movie2", "movie3"]
}

# Item popularity
item_popularity = {
    "movie1": 5,
    "movie2": 10,
    "movie3": 7,
    "movie4": 15
}
```

Variables Terminal

Recommendation_System_Ragini.ipynb

File Edit View Insert Runtime Tools Help

Commands Code Text Run all

Connect

BATCH PROCESSING

```
1) def batch_recommend(user):
    watched = user_history.get(user, [])
    recommendations = []

    for item in item_popularity:
        if item not in watched:
            recommendations.append((item, item_popularity[item]))

    recommendations.sort(key=lambda x: x[1], reverse=True)
    return [item for item, _ in recommendations[:3]]

print("Batch Recommendation:", batch_recommend("user1"))
```

Batch Recommendation: ['movie4', 'movie3']

REAL-TIME PROCESSING

```
1) def real_time_update(user, new_item):
    if user not in user_history:
        user_history[user] = []

    user_history[user].append(new_item)

    if new_item in item_popularity:
        item_popularity[new_item] += 1
    else:
        item_popularity[new_item] = 1

# simulate real-time
```

Variables Terminal

Recommendation_System_Ragini.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

Connect

QUEUE (Kafka Simulation)

```
[ ]
from queue import Queue

event_queue = Queue()

def send_event(user, item):
    event_queue.put((user, item))

def process_events():
    while not event_queue.empty():
        user, item = event_queue.get()
        real_time_update(user, item)

# simulate event
send_event("user2", "movie4")
process_events()

print("Queue Processed:", batch_recommend("user2"))
```

Queue Processed: ['movie1']

Recommendation_System_Ragini.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

Connecting

CACHE (Redis Simulation)

```
[ ]
cache = {}

def get_recommendation(user):
    if user in cache:
        print("Cache Hit")
        return cache[user]

    print("Cache Miss")
    result = batch_recommend(user)
    cache[user] = result
    return result

print(get_recommendation("user1"))
print(get_recommendation("user1"))
```

... Cache Miss
['movie3']
Cache Hit
['movie3']

Variables

Terminal

Connecting to Python 3

Explanation:

- User history is stored as batch data
 - Item popularity is used for ranking
 - Real-time updates simulate user interactions
 - Recommendations are generated based on combined logic
-

8. Key Concepts Used

- Client-Server Architecture
 - Queue (Event-driven system)
 - Batch Processing
 - Real-time Processing
 - Caching
 - Ranking Algorithm
 - Scalability
-

9. Real-Life Use Case

- Netflix → Movie recommendations
 - Amazon → Product suggestions
 - Instagram → Feed ranking
-

10. Project Files

- Google Colab Notebook (.ipynb)
 - Architecture Diagram
 - Output Screenshots
-

11. GitHub Repository

Example:

<https://github.com/RaginiSingh2024/recommendation-system-batch-realtime>

12. Conclusion

This project demonstrates how modern systems combine:

- **Batch Processing** → for accuracy
- **Real-time Processing** → for freshness
- **Cache** → for fast response
- **Queue** → for scalability

to deliver efficient and personalized recommendations.

13. Future Improvements

- Add Machine Learning models
 - Use real database (MongoDB / SQL)
 - Build APIs using Flask/FastAPI
 - Deploy system on cloud
 - Improve ranking algorithm
-